Introduction to AI

Last updated: 2026-05-18 18:46:28+03:00

Contents

This document has been partly produced using AI. See the following log on the interaction with OpenAI Codex: `https://github.com/SamiSieranoja/aint/blob/main/text/codex-log.txt`

Chapter 1

Artificial Intelligence

1.1 Definition of Artificial Intelligence

Artificial intelligence (AI) is the study and construction of computational systems that perform tasks which, when carried out by humans, are associated with intelligence. Such tasks include perception, language use, reasoning, planning, learning, decision-making, and acting in complex environments.

This view follows the original formulation in the 1955 Dartmouth proposal by John McCarthy and his co-authors, where the AI problem was described as "making a machine behave in ways that would be called intelligent if a human were so behaving."

1.2 Difficulties in Defining AI

Defining AI is difficult because intelligence itself is not a single, precisely measurable property. Human intelligence includes many abilities, and these abilities do not map cleanly onto computational difficulty.

One important observation is that *tasks easy for humans may be difficult for computers*, while *tasks difficult for humans may be easy for computers*. For example, multiplications with large numbers is hard and slow for most humans but straightforward for computers. In contrast, recognizing faces, understanding ambiguous speech, or moving through an unstructured environment are routine for humans but technically demanding for machines. This contrast is often called Moravec's paradox.[1]

Another problem is that the boundary of AI changes over time. Once a task is solved reliably, it often stops being perceived as AI and becomes ordinary software engineering. Optical character recognition, route planning, spam filtering, and speech recognition illustrate this shift.

There is also no single criterion for success. Some definitions emphasize human-like behavior, as in the Turing test. Others emphasize rational behavior: the system should choose actions that are appropriate for achieving its goals given its information and constraints. These perspectives lead to different research questions and different evaluation methods.

1.3 Narrow and General AI

Narrow AI, also called weak AI, refers to systems designed to perform specific tasks or classes of tasks. Examples include a chess engine, a medical image classifier, a recommendation system, or a

[1] `https://en.wikipedia.org/wiki/Moravec%27s_paradox`

speech recognizer. Such systems may exceed human performance within their domain, but their competence remains limited to that specific area.

General AI, often called artificial general intelligence (AGI), refers to a hypothetical system with broad, flexible intellectual competence across many domains. A general AI would not merely perform one predefined task; it would be able to learn new tasks, transfer knowledge between domains, reason about novel situations, and adapt to changing environments with human-level or greater generality.

The distinction is important because narrow systems can be highly capable without being generally intelligent. A system may translate text, classify images, or solve mathematical problems, yet still lack robust common sense, long-term autonomy, or reliable transfer to unfamiliar settings.

Current AI systems are still considered as narrow AI. However, there is a general trend of AI systems becoming more and more general[2].

1.4 Self-Improvement and the Singularity

Self-improvement in AI refers to the ability of an AI system to improve its own performance. In a limited sense, this already occurs in machine learning: systems update parameters from data, tune policies through reinforcement learning, or use search to improve solutions. Usually, however, this occurs within boundaries set by human designers, such as a fixed architecture, training procedure, objective, and data source.

Recursive self-improvement is a stronger idea. It describes a system that can modify its own algorithms, architecture, training process, or hardware usage in ways that increase its ability to make further improvements. If each improvement made the next improvement easier, capability growth could in principle accelerate.

In ordinary technological progress, humans remain in the loop: they set goals, design systems, test them, and decide what to deploy. The technological *singularity* concerns a more advanced situation, where AI systems could take over some of these roles and accelerate their own development. If this process became fast enough, it might become difficult for humans to understand, predict, or control.

An early formulation of this idea appears in I.J. Good's essay *Speculations Concerning the First Ultraintelligent Machine* (1965). Good argued that a machine able to surpass humans in all intellectual activities could also design better machines, producing an "intelligence explosion." He therefore described the first ultraintelligent machine as potentially the last invention humans would need to make, provided that it remained controllable.

[2] `https://www.noemamag.com/artificial-general-intelligence-is-already-here/`

Chapter 2

Neural Networks

In this chapter, we'll discuss:

- Neural networks, beginning with the simpler model of linear regression, and how an artificial neuron can be constructed by adding an activation function to a linear model.
- How individual neurons are combined into layers and how layers form neural networks.
- Matrix-vector multiplication and how it provides a compact way to represent neural network computations.

2.1 Linear Regression

Linear regression is a basic supervised learning method for predicting a numeric output from input features. Given an input vector $x = (x_1, \ldots, x_n)$, a linear regression model computes

$$\hat{y} = w_1 x_1 + \cdots + w_n x_n + b,$$

where $w_1, \ldots, w_n$ are weights and b is a bias term. The model is linear in the input features: changing an input changes the prediction by a fixed amount determined by its weight.

During training, the weights and bias are chosen to minimize prediction error on examples. A common objective is mean squared error, which penalizes the squared difference between the predicted value $\hat{y}$ and the target value y.

2.2 A Single Neuron

A single artificial neuron starts with the same weighted sum used in linear regression:

$$z = w_1 x_1 + \cdots + w_n x_n + b.$$

The difference is that the neuron then applies an activation function f:

$$a = f(z).$$

The output a is called the activation of the neuron.

Without the activation function, a neuron is only a linear model.

There are several types of activation functions, but we'll mention the ReLU function first because it is one of the most simple and also very commonly used. It has three common definitions:

$$\mathrm{ReLU}(z) = \max(0, z) = \frac{z + |z|}{2} = \begin{cases} 0, & z < 0, \\ z, & z \geq 0 \end{cases}$$

ReLU returns zero for negative inputs and returns the input itself for positive inputs. It is simple, computationally cheap, and widely used in hidden layers of neural networks.

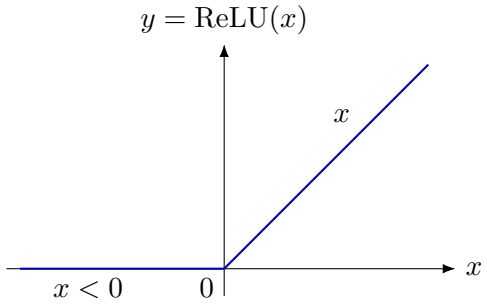

Figure 2.1: ReLU maps negative inputs to zero and leaves positive inputs unchanged.

2.3 Examples of a Single Neuron

Consider a neuron with three inputs, weights w_1, w_2, w_3, bias b, and ReLU activation $\sigma(z) = \mathrm{ReLU}(z)$. Its output is

$$y = \sigma(w_1x_1 + w_2x_2 + w_3x_3 + b).$$

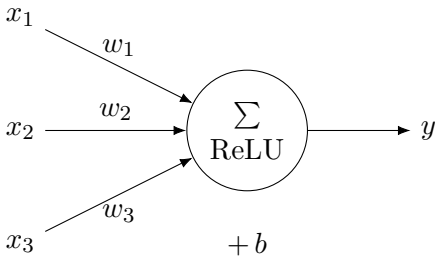

Figure 2.2: A single neuron computes a weighted sum, adds a bias, and applies ReLU.

Let $w_1 = 4$, $w_2 = 1$, $w_3 = -2$, and $b = -4$. For $x_1 = 3$, $x_2 = 2$, and $x_3 = 1$,

$$y = \sigma(4 \cdot 3 + 1 \cdot 2 + (-2) \cdot 1 - 4) = \sigma(8) = 8.$$

For $x_1 = 1$, $x_2 = 6$, and $x_3 = 5$,

$$y = \sigma(4 \cdot 1 + 1 \cdot 6 + (-2) \cdot 5 - 4) = \sigma(-4) = 0.$$

The two examples use the same neuron parameters. Only the input changes, so the weighted sum changes from a positive value to a negative value; ReLU passes the positive value through and clips the negative value to zero.

2.4 Dot Products

The weighted sum in linear regression and in a neuron can be written as a dot product. For two vectors

$$w = (w_1, \ldots, w_n), \qquad x = (x_1, \ldots, x_n),$$

their dot product is

$$w \cdot x = w_1 x_1 + \cdots + w_n x_n.$$

Thus the pre-activation value of a neuron can be written compactly as

$$z = w \cdot x + b.$$

The dot product measures a weighted combination of the input features.

For example, if $a = (1, 2, 3)$ and $b = (4, 5, 6)$, then

$$a \cdot b = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 4 + 10 + 18 = 32.$$

2.5 Matrix-Vector Multiplication

A layer of several neurons can be represented by a weight matrix. Suppose a layer has m neurons and each neuron receives the same input vector x with n features. The weights can be arranged as a matrix

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix}.$$

The product Wx is a vector with one entry for each neuron:

$$Wx = \begin{bmatrix} w_1 \cdot x \\ w_2 \cdot x \\ \vdots \\ w_m \cdot x \end{bmatrix}.$$

Matrix-vector multiplication is therefore repeated dot product: The matrix is split into rows and then we calculate the dot product of the input vector and each row of the matrix.

For example, let

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad x = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}.$$

Then

$$Ax = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 2 + 3 \cdot (-1) \\ 4 \cdot 1 + 5 \cdot 2 + 6 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 2 \\ 8 \end{bmatrix}.$$

2.6 Layer Calculations as Matrix Operations

Using matrices, the computation of an entire layer can be written as

$$z = Wx + b,$$

where b is now a vector of bias terms. The activation function is then applied component by component:

$$a = f(z).$$

For ReLU, this means applying $\max(0, z_i)$ to each component z_i.

A neural network consists of several such layers. If $a^{(0)} = x$ is the input, then a typical layer computes

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = f(z^{(\ell)}).$$

This notation shows why matrix-vector multiplication is central to neural networks: it computes all weighted sums in a layer at once. Modern hardware can perform these matrix operations efficiently, which is one reason neural networks scale to large models and large datasets.

2.7 Example: A Two-Layer Network

Consider a network with input

$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \qquad W_1 = \begin{bmatrix} 1 & 4 \\ 0 & -3 \end{bmatrix}, \qquad W_2 = \begin{bmatrix} -2 & 7 \\ 3 & -9 \end{bmatrix}.$$

Use bias vectors

$$b^{(1)} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}, \qquad b^{(2)} = \begin{bmatrix} 3 \\ 4 \end{bmatrix},$$

and apply ReLU after each layer.

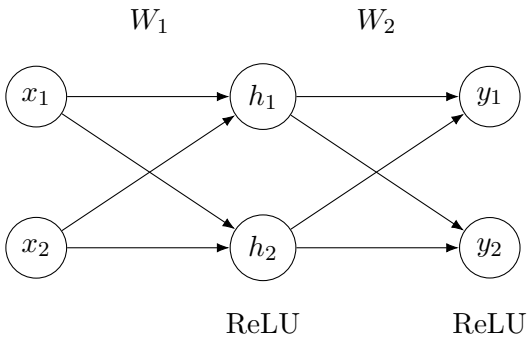

Figure 2.3: A two-layer fully connected neural network with two input units, two hidden units, and two output units.

The first layer computes

$$z^{(1)} = W_1 x + b^{(1)} = \begin{bmatrix} 1 & 4 \\ 0 & -3 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \begin{bmatrix} -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 2 + 4 \cdot 3 - 2 \\ 0 \cdot 2 + (-3) \cdot 3 + 1 \end{bmatrix} = \begin{bmatrix} 12 \\ -8 \end{bmatrix}.$$

After ReLU,

$$a^{(1)} = \text{ReLU}(z^{(1)}) = \begin{bmatrix} 12 \\ 0 \end{bmatrix}.$$

The second layer computes

$$\begin{aligned} z^{(2)} &= W_2 a^{(1)} + b^{(2)} \\ &= \begin{bmatrix} -2 & 7 \\ 3 & -9 \end{bmatrix} \begin{bmatrix} 12 \\ 0 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} \\ &= \begin{bmatrix} (-2) \cdot 12 + 7 \cdot 0 + 3 \\ 3 \cdot 12 + (-9) \cdot 0 + 4 \end{bmatrix} \\ &= \begin{bmatrix} -21 \\ 40 \end{bmatrix}. \end{aligned}$$

Applying ReLU again gives the network output

$$y = \text{ReLU}(z^{(2)}) = \begin{bmatrix} 0 \\ 40 \end{bmatrix}.$$

2.8 Summary

Linear regression predicts a numeric value using a weighted sum of input features. A neuron generalizes this idea by applying an activation function to the weighted sum. ReLU is a simple nonlinear activation that returns zero for negative inputs and the input itself for positive inputs. Dot products express weighted sums compactly, and matrix-vector multiplication computes many dot products at once. Neural network layers are therefore naturally represented as matrix-vector calculations followed by nonlinear activation functions.

Chapter 3

Gradient Descent